



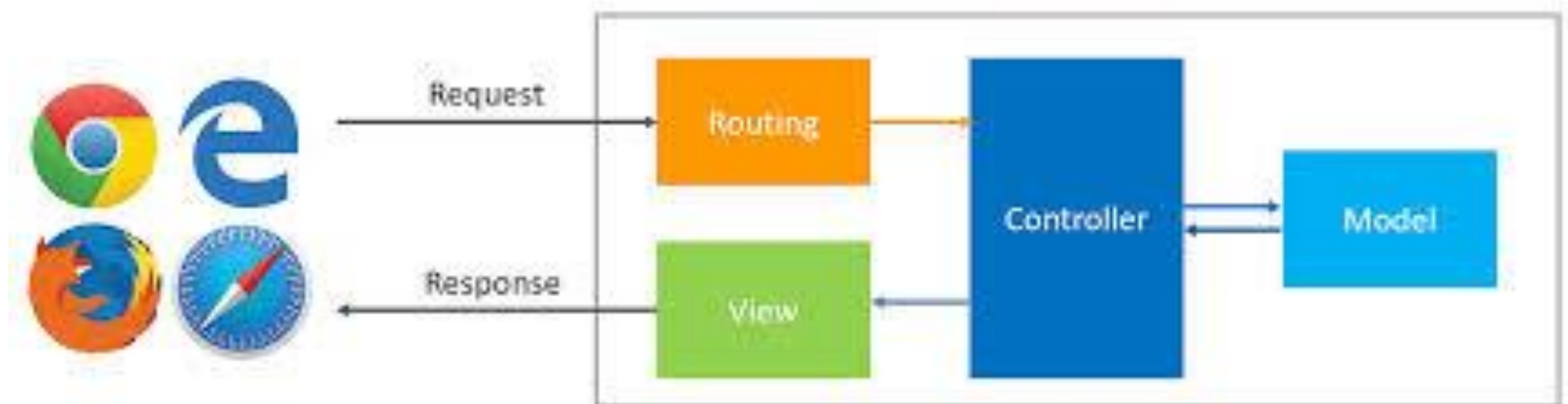
CMC UNIVERSITY

Aspire to Inspire the Digital World

Controller trong MVC

Khoa CNTT&TT, Trường Đại học CMC

Hà Nội, tháng 12 năm 2025



- Trong **ASP.NET MVC**, *Controller* là một thành phần trung tâm chịu trách nhiệm xử lý yêu cầu từ người dùng, tương tác với dữ liệu (Model), và trả về kết quả hiển thị (View).
- Trong mô hình Model-View-Controller, Controller đóng vai trò “người điều phối”:
 - Nhận request từ client (trình duyệt)
 - Xử lý logic nghiệp vụ (business logic)
 - Gọi Model để lấy / cập nhật dữ liệu
 - Trả về View hoặc dữ liệu (JSON, file, ...)

- Trong ASP.NET MVC, Controller là một class:
 - Kế thừa từ Controller
 - Tên phải kết thúc bằng Controller

```
</> C#  
  
public class HomeController : Controller  
{  
    public ActionResult Index()  
    {  
        return View();  
    }  
}
```

- Là các hàm bên trong Controller
- Mỗi Action tương ứng với một URL

```
public ActionResult About()  
{  
    return View();  
}
```

- /Home/About => gọi method About()

1. User gửi request (URL)
2. Routing xác định Controller + Action
3. Controller:
 1. Xử lý logic
 2. Gọi Model
4. Trả về:
 1. View (HTML)
 2. JSON (API)
 3. Redirect, File, ...

- Trả về View

```
return View();
```

- Trả về JSON

```
return Json(data);
```

- Redirect

```
return RedirectToAction("Index");
```

- Trả về nội dung trực tiếp

```
return Content("Hello");
```

- ViewBag

```
ViewBag.Message = "Hello";
```

- ViewData

```
ViewData["Message"] = "Hello";
```

- Model (khuyến nghị)

```
return View(model);
```



```
public ActionResult Index()
{
    var students = new List<Student>
    {
        new Student { Id = 1, Name = "An", Age = 20 },
        new Student { Id = 2, Name = "Bình", Age = 21 }
    };

    return View(students);
}
```

```
</> HTML

@model List<Student>

<h2>Danh sách sinh viên</h2>

@foreach (var s in Model)
{
    <p>@s.Name - @s.Age</p>
}
```

- @model = khai báo kiểu dữ liệu
- Model = biến chứa dữ liệu
- 👉 Nếu sai kiểu → lỗi compile ngay

```
ViewBag.Message = "Hello sinh viên";  
ViewBag.Total = 100;
```

</> HTML

```
<h3>@ViewBag.Message</h3>  
<p>@ViewBag.Total</p>
```

! Nhược điểm

- Không có type checking
- Sai tên → chạy lỗi runtime

@ViewBag.Messag // ❌ sai mà không báo compile

```
ViewData["Message"] = "Hello";
```

</> HTML

```
<h3>@ViewData["Message"]</h3>
```

! Nhược điểm

- Phải cast kiểu
- Khó maintain

Tiêu chí	Model	ViewBag	ViewData
Type-safe	✓	✗	✗
Dễ dùng	✓	✓	✗
Khuyến nghị	★ ★ ★ ★ ★	★ ★	★

- 1 object

```
</> C#  
  
return View(student);
```

```
</> HTML  
  
@model Student  
  
<h2>@Model.Name</h2>
```

- List object

```
</> C#  
  
return View(students);
```

```
</> HTML  
  
@model List<Student>
```

- Cách 1: Dùng ViewModel (chuẩn nhất)

```
public class StudentViewModel
{
    public List<Student> Students { get; set; }
    public string Keyword { get; set; }
}
```

```
var vm = new StudentViewModel
{
    Students = _service.GetAll(),
    Keyword = "An"
};

return View(vm);
```

</> HTML

```
@model StudentViewModel

<p>Từ khóa: @Model.Keyword</p>

@foreach (var s in Model.Students)
{
    <p>@s.Name</p>
}
```

- Cách 2: Trộn Model + ViewBag (không khuyến khích)

```
ViewBag.Keyword = "An";  
return View(students);
```



CMC UNIVERSITY

Aspire to Inspire the Digital World



Thực hành



- Project: Student Management Lite (CRUD + Search + API)

- Sau project này, sinh viên sẽ:
 - Hiểu rõ vai trò của Controller trong Model-View-Controller
 - Phân biệt:
 - Controller vs Service vs Model
 - Biết cách:
 - Nhận request → xử lý → trả response
 - Thực hành đầy đủ:
 - CRUD
 - Routing
 - Return View / JSON / Redirect

- Xây dựng hệ thống quản lý sinh viên đơn giản:
- **Chức năng:**
 - Xem danh sách sinh viên
 - Thêm sinh viên
 - Sửa thông tin
 - Xóa sinh viên
 - Tìm kiếm theo tên
 - API trả JSON danh sách sinh viên

- Dùng ASP.NET MVC

```
/Controllers
    StudentController.cs

/Models
    Student.cs

/Services
    IStudentService.cs
    StudentService.cs

/Views/Student
    Index.cshtml
    Create.cshtml
    Edit.cshtml
```

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Age { get; set; }
}
```

```
public interface IStudentService
{
    List<Student> GetAll();
    Student GetById(int id);
    void Add(Student student);
    void Update(Student student);
    void Delete(int id);
    List<Student> Search(string keyword);
}
```

```
public class StudentService : IStudentService
{
    private static List<Student> _students = new List<Student>();

    public List<Student> GetAll() => _students;

    public Student GetById(int id) => _students.FirstOrDefault(x => x.Id == id);

    public void Add(Student student)
    {
        student.Id = _students.Count + 1;
        _students.Add(student);
    }

    public void Update(Student student)
    {
        var s = GetById(student.Id);
        if (s != null)
        {
            s.Name = student.Name;
            s.Age = student.Age;
        }
    }

    public void Delete(int id)
    {
        var s = GetById(id);
        if (s != null) _students.Remove(s);
    }

    public List<Student> Search(string keyword)
    {
        return _students
            .Where(x => x.Name.Contains(keyword))
            .ToList();
    }
}
```



```
public class StudentController : Controller
{
    private readonly IStudentService _service;

    public StudentController()
    {
        _service = new StudentService();
    }

    // 1. GET: /Student
    public ActionResult Index(string keyword)
    {
        var data = string.IsNullOrEmpty(keyword)
            ? _service.GetAll()
            : _service.Search(keyword);

        return View(data);
    }

    // 2. GET: Create
    public ActionResult Create()
    {
        return View();
    }

    // 3. POST: Create
    [HttpPost]
    public ActionResult Create(Student student)
    {
        _service.Add(student);
        return RedirectToAction("Index");
    }
}
```

```
// 4. GET: Edit
public ActionResult Edit(int id)
{
    var student = _service.GetById(id);
    return View(student);
}

// 5. POST: Edit
[HttpPost]
public ActionResult Edit(Student student)
{
    _service.Update(student);
    return RedirectToAction("Index");
}

// 6. Delete
public ActionResult Delete(int id)
{
    _service.Delete(id);
    return RedirectToAction("Index");
}

// 7. API JSON (để thấy Controller trả nhiều kiểu)
public JsonResult GetAllJson()
{
    return Json(_service.GetAll(), JsonRequestBehavior.AllowGet);
}
}
```

```
@model List<Student>
```

```
<form method="get">  
  <input type="text" name="keyword" placeholder="Search..." />  
  <button type="submit">Search</button>  
</form>
```

```
<a href="/Student/Create">Add New</a>
```

```
<table>  
  <tr>  
    <th>Name</th>  
    <th>Age</th>  
    <th>Action</th>  
  </tr>
```

```
@foreach (var s in Model)  
{  
  <tr>  
    <td>@s.Name</td>  
    <td>@s.Age</td>  
    <td>  
      <a href="/Student/Edit/@s.Id">Edit</a>  
      <a href="/Student/Delete/@s.Id">Delete</a>  
    </td>  
  </tr>  
}  
</table>
```



- Controller làm gì trong từng function?
- Tại sao không viết logic trong Controller?
- Nếu bỏ Service thì chuyện gì xảy ra?
- Action nào:
 - Trả View?
 - Trả JSON?
 - Redirect?

- Vấn đề hiện tại (chưa dùng DI)
 - Controller **phụ thuộc chặt** vào implementation
 - Không test/mock được
 - Khó thay đổi (ví dụ đổi sang DB thật)

```
public class StudentController : Controller
{
    private readonly IStudentService _service;

    public StudentController()
    {
        _service = new StudentService(); // ❌ hard-code
    }
}
```

- Không tự tạo object → để framework “inject” vào

Bước 1: Đăng ký Service trong Program.cs

```
</> C#  
  
var builder = WebApplication.CreateBuilder(args);  
  
// Đăng ký DI  
builder.Services.AddScoped<IStudentService, StudentService>();  
builder.Services.AddScoped<IStudentRepository, StudentRepository>();  
  
var app = builder.Build();
```

Bước 2: Inject vào Controller

```
</> C#  
  
public class StudentController : Controller  
{  
    private readonly IStudentService _service;  
  
    public StudentController(IStudentService service)  
    {  
        _service = service; // ✅ được inject  
    }  
}
```

Bước 3: Inject tiếp vào Service

```
</> C#  
  
public class StudentService : IStudentService  
{  
    private readonly IStudentRepository _repo;  
  
    public StudentService(IStudentRepository repo)  
    {  
        _repo = repo;  
    }  
}
```



CMC UNIVERSITY

Aspire to Inspire the Digital World



THANK YOU